

Go Variadic Functions: Complete Deep Dive

Part 1: What Variadic Really Is (No Magic)

The Basic Truth

```
go
func f(x int, args ...int)
```

Inside this function:

```
go
// args is just a regular slice
// Type: []int
// Nothing special, nothing magical
```

Test it:

```
go
func test(args ...int) {
    fmt.Printf("Type: %T\n", args) // []int
    fmt.Printf("Value: %v\n", args) // [1 2 3]

    // Everything that works on []int works here:
    len(args)
    cap(args)
    args[0] = 999
    args = append(args, 5)
    for _, v := range args { }
}

test(1, 2, 3)
```

Key insight: No special type. No runtime magic. Just a slice.

Part 2: How the Compiler Transforms Your Code

Form 1: You Pass Individual Arguments

You write:

```
go  
f(1, 2, 3, 4)
```

Compiler transforms to:

```
go  
  
tmp := []int{2, 3, 4} // Allocate new slice  
f(1, tmp...)         // Pass it
```

What this means:

- Go creates a slice for you
- Copies your arguments into it
- This is an **allocation** (costs memory)

Visual:

```
Your code:    f(1, 2, 3, 4)  
              ↓  
Compiler does: tmp := []int{2, 3, 4} ← Allocation happens here  
              ↓  
Really calls:  f(1, tmp...)
```

Form 2: You Pass an Existing Slice

You write:

```
go  
  
s := []int{2, 3, 4}  
f(1, s...)
```

Compiler does:

```
go
```

```
// NO new allocation
// Just passes the slice header directly
f(1, s)
```

Why is this cheaper?

- No memory allocation
- No copying of data
- Just passes the existing slice
- Much faster

Comparison:

```
go

// EXPENSIVE - allocates every call
for i := 0; i < 1000000; i++ {
    sum(1, 2, 3, 4, 5) // Creates []int{1,2,3,4,5} 1 million times!
}

// CHEAP - allocates once
args := []int{1, 2, 3, 4, 5}
for i := 0; i < 1000000; i++ {
    sum(args...) // Reuses same slice
}
```

Part 3: Hidden Allocation Cost (Performance Trap!)

The Problem

```
go

func log(level string, args ...Interface{}) {
    // logging logic
}

// In a hot loop:
for i := 0; i < 1000000; i++ {
    log("INFO", i, i+1, i+2) // Allocation happens 1 million times!
}
```

Each call does:

1. Allocate a new `[]interface{}` slice
2. Copy arguments into it
3. Box each `int` into `interface{}`
4. Pass to function

Cost:

- 1 million allocations
- GC pressure (garbage collector works harder)
- CPU time wasted on allocations

Why This Matters

`fmt.Printf` is expensive because of this:

```
go
fmt.Printf("x=%d y=%d z=%d\n", x, y, z)
// Creates []interface{}{x, y, z} every single time
```

Performance-critical code avoids this:

```
go
// Instead of variadic:
func log(args ...string)

// Use regular slice:
func log(args []string)
```

Then caller can reuse:

```
go
```

```
buffer := make([]string, 3) // Allocate once
for i := 0; i < 1000000; i++ {
    buffer[0] = "INFO"
    buffer[1] = fmt.Sprint(i)
    buffer[2] = "message"
    log(buffer) // No allocation in loop!
}
```

Part 4: Why Go Has Variadic (Design Decision)

Reason 1: Clean API (Ergonomics)

Without variadic:

```
go
fmt.Println([]interface{}{"Hello", "World", 123}) // Ugly!
append(s, []int{1, 2, 3})                        // Awkward!
```

With variadic:

```
go
fmt.Println("Hello", "World", 123) // Natural!
append(s, 1, 2, 3)                 // Beautiful!
```

Reason 2: Flexible Call Sites

```
go
func buildURL(parts ...string) string {
    // Can handle any number of parts
}

buildURL("api", "users")           // 2 parts
buildURL("api", "users", "123")    // 3 parts
buildURL("api", "users", "123", "posts") // 4 parts
```

Without variadic, you'd need multiple functions or always pass a slice.

Reason 3: Avoid Manual Slice Building

User wants to print multiple things:

With variadic:

```
go
fmt.Println(a, b, c, d, e) // Easy!
```

Without variadic:

```
go
fmt.Println([]interface{}{a, b, c, d, e}) // Annoying
```

Design philosophy: API convenience over performance.

Part 5: Why Only Last Parameter Can Be Variadic

The Rule

```
go
func f(a ...int, b int) // ✗ ILLEGAL - compile error
func f(b int, a ...int) // ✓ LEGAL
```

Why This Rule Exists

Imagine if you could put variadic anywhere:

```
go
func f(a ...int, b int)

// How would you call it?
f(1, 2, 3, 4, 5)
```

Which elements go where?

- Is it: `a = [1,2,3,4]`, `b = 5`?
- Or: `a = [1]`, `b = 2`, and what about 3,4,5?

- Or: `a = [1,2,3]`, `b = 4`, and what about 5?

Ambiguous! Parser can't decide.

The Solution

Only allow variadic at the end:

```
go

func f(b int, a ...int)

f(1, 2, 3, 4, 5)
// Clear: b=1, a=[2,3,4,5]
```

Simple grammar, no ambiguity.

Part 6: Zero Arguments Case

You Can Call With No Variadic Arguments

```
go

func greet(prefix string, names ...string) {
    fmt.Printf("Length: %d\n", len(names))
    fmt.Printf("Value: %v\n", names)
    fmt.Printf("Is nil? %v\n", names == nil)
}

greet("Hello")
```

Output:

```
Length: 0
Value: []
Is nil? true
```

Important Discovery: It's Actually `nil`!

Not an empty slice, but actual `nil`:

```
go
```

```
func test(args ...int) {  
    if args == nil {  
        fmt.Println("Got nil!") // This prints!  
    }  
}  
  
test() // Called with no arguments
```

Comparison:

```
go  
  
var a []int = nil    // nil slice  
var b []int = []int{} // empty slice  
  
a == nil // true  
b == nil // false  
len(a)   // 0  
len(b)   // 0
```

Why does this matter?

You can detect "no arguments passed":

```
go  
  
func process(items ...string) {  
    if items == nil {  
        fmt.Println("User didn't pass anything")  
    } else if len(items) == 0 {  
        fmt.Println("User passed empty slice explicitly")  
    }  
}  
  
process()           // nil case  
process([]string{}...) // empty slice case
```

Part 7: You Cannot Mix Explicit and Variadic Unpacking

This Doesn't Work

```
go
s := []int{2, 3, 4}

f(1, s..., 5) // ✗ COMPILE ERROR
```

Why Not?

After `s...`, where does the variadic part end?

```
go
f(1, s..., 5)
↓
Does this unpack to: f(1, 2, 3, 4, 5)?
Or does compiler think 5 is separate?
```

Ambiguous grammar!

The Rule: `...` Must Be The Last Thing

```
go

// ✓ OK
f(s...)
f(1, s...)

// ✗ ILLEGAL
f(s..., 1)
f(1, s..., 2)
```

Keeps compiler simple, avoids parsing ambiguity.

Part 8: Passing `nil` vs Empty Slice

Subtle Type System Issue

This is INVALID:

```
go
```

```
f(1, nil...) // ✗ ERROR: cannot use untyped nil
```

Why? Because `nil` has no type, and Go doesn't know what slice type to make.

This is VALID:

```
go
```

```
f(1, []int(nil)...) // ✓ OK: typed nil
```

```
f(1, []int{}...) // ✓ OK: empty slice
```

```
var s []int = nil
```

```
f(1, s...) // ✓ OK: variable has type
```

The Difference

```
go
```

```
func test(args ...int) {  
    fmt.Printf("Is nil? %v\n", args == nil)  
    fmt.Printf("Length: %d\n", len(args))  
}
```

```
test([]int(nil)...) // Is nil? true, Length: 0
```

```
test([]int{}...) // Is nil? false, Length: 0
```

```
test() // Is nil? true, Length: 0
```

All behave the same in loops, but `nil` check differs!

Part 9: Variadic + `interface{}` = Double Boxing Cost

The Most Expensive Pattern

```
go
```

```
func f(args ...interface{})
```

When you call:

```
go
```

```
f(1, 2, 3)
```

Go does TWO expensive things:

Step 1: Create slice

```
go  
  
tmp := []interface{}{?, ?, ?} // Allocate slice
```

Step 2: Box each argument

```
go  
  
tmp[0] = interface{}(1) // Box int to interface  
tmp[1] = interface{}(2) // Box int to interface  
tmp[2] = interface{}(3) // Box int to interface
```

Visual Breakdown

```
Your code:  f(1, 2, 3)  
            ↓  
Step 1:    Allocate []interface{} with cap=3  
            ↓  
Step 2:    Box 1 → interface{} (heap allocation)  
           Box 2 → interface{} (heap allocation)  
           Box 3 → interface{} (heap allocation)  
            ↓  
Step 3:    Store boxed values in slice  
            ↓  
Step 4:    Call f(slice)
```

Total cost:

- 1 slice allocation
- 3 interface boxing allocations
- 4 allocations for 3 simple integers!

Why `fmt.Println` Is Slow

```
go
```

```
fmt.Println(a, b, c, d, e)
// Signature: func Println(a ...interface{})
//
// Creates: []interface{} with 5 elements
// Boxes: Each of a, b, c, d, e into interface{}
// Total: 6 allocations minimum
```

In hot loops, this kills performance.

The Alternative

```
go

// Instead of:
func log(args ...interface{})

// Consider:
func log(args []string) // Pre-format to strings
func logInt(args []int) // Type-specific
```

Part 10: `append` Is Special (Compiler Magic)

`append` Is Not a Normal Function

```
go

append(s, 1, 2, 3)
append(s, other...)
```

`append` is a compiler intrinsic:

- Not defined in normal Go code
- Compiler has special lowering rules
- Can optimize allocations away
- Can inline aggressively

Normal Variadic vs `append`

Normal variadic function:

```
go
```

```
func sum(nums ...int) int {  
    // Always allocates slice at call site  
}
```

```
sum(1, 2, 3) // Allocation happens
```

append (special):

```
go
```

```
s = append(s, 1, 2, 3)  
// Compiler can:  
// - Avoid slice allocation if cap is sufficient  
// - Inline the entire operation  
// - Optimize into direct memory writes
```

Why the difference?

- **append** is built into the compiler
- It's one of the few built-in variadic optimizations
- Normal functions don't get this treatment

You Can't Replicate This

```
go
```

```
// You can't write your own append-like function  
// that gets the same optimizations  
func myAppend(s []int, values ...int) []int {  
    // This will always allocate 'values' slice  
    return append(s, values...)  
}
```

Part 11: You Cannot Take Address of Variadic Parameter (Carefully)

What You CAN Do

```
go
```

```
func f(args ...int) {  
    p := &args    // ✓ OK: address of slice header  
  
    if len(args) > 0 {  
        q := &args[0] // ✓ OK: address of first element  
    }  
}
```

What This Means

The slice header itself:

```
go  
  
p := &args // Points to the slice structure (ptr, len, cap)
```

Elements in the slice:

```
go  
  
q := &args[0] // Points to actual data in underlying array
```

Why This Matters: Escape Analysis

The slice might escape to heap:

```
go  
  
func f(args ...int) *int {  
    return &args[0] // args must escape to heap  
}
```

You cannot assume stack layout:

- `args` might be on heap
- Underlying array might be on heap
- No guarantees about contiguous memory

Treat as normal slice, no special assumptions.

Part 12: Variadic Forwarding - The `(...)` Rule

To Forward Variadic Arguments

```
go

func f(args ...int) {
    g(args...) // ✓ MUST use ... to forward
}

func g(nums ...int) {
    // receives the arguments
}
```

Common Mistake

```
go

func f(args ...int) {
    g(args) // ✗ COMPILE ERROR
}

func g(nums ...int) {
}
```

Error:

```
cannot use args (type []int) as type int in argument to g
```

Why?

`g` expects variadic parameters, not a slice:

```
go

g(1, 2, 3) // ✓ OK
g([]int{1,2,3}) // ✗ ERROR: expects int, got []int
g([]int{1,2,3}...) // ✓ OK: ... unpacks the slice
```

The Rule: `(...)` Is an Instruction to Compiler

```
args... means "unpack this slice into individual arguments"
```

Visual:

```
go

func f(args ...int) {
    g(args...)
}

// When f receives [1, 2, 3]:
args = [1, 2, 3]
    ↓
g(args...) unpacks to: g(1, 2, 3)
    ↓
g receives [1, 2, 3] in its variadic parameter
```

Forwarding Chain Example

```
go

func level1(nums ...int) {
    level2(nums...) // Forward to level2
}

func level2(nums ...int) {
    level3(nums...) // Forward to level3
}

func level3(nums ...int) {
    fmt.Println(nums)
}

level1(1, 2, 3) // Prints: [1 2 3]
```

Part 13: Variadic Is Mostly Compile-Time (Shocking)

The Truth About Runtime

At runtime, variadic functions don't exist as a concept.

After compilation:

- No special "variadic calling convention"

- Functions just receive slices
- All the magic happens during compilation

Proof

These are identical at runtime:

```
go

// Variadic version
func sum(nums ...int) int

// Regular slice version
func sum(nums []int) int
```

Both receive the exact same thing: a slice.

What the Compiler Does

```
go

// You write:
sum(1, 2, 3)

// Compiler compiles to (pseudo-code):
tmp := makeSlice(int, 3)
tmp[0] = 1
tmp[1] = 2
tmp[2] = 3
sum(tmp) // Just a regular function call with a slice
```

After compilation:

- No variadic concept exists
- Just slice passing
- Normal function call

Why This Matters

Debugging: When you look at assembly or memory, you just see slice operations.

Performance: Compiler can't optimize variadic calls specially (except `append`).

Understanding: Variadic is syntactic sugar, not a runtime feature.

Part 14: Real Drawbacks (Production Concerns)

Drawback 1: Hidden Allocations

```
go

func log(msg string, args ...interface{}) {
    // Looks innocent
}

// But every call allocates:
log("Error", err)      // 1 allocation
log("Values", a, b, c) // 1 allocation
log("Data", x, y, z, w) // 1 allocation
```

In a hot loop:

```
go

for i := 0; i < 1000000; i++ {
    log("Iteration", i) // 1 million allocations!
}
```

Drawback 2: Harder to Optimize

Compiler struggles to:

- Inline variadic functions (harder than regular functions)
- Eliminate bounds checks
- Optimize escape analysis

Regular function:

```
go

func process(items []int) {
    // Easier for compiler to optimize
}
```

Drawback 3: Interface Boxing Overhead

```
go
func f(args ...interface{})
```

Every call boxes all arguments:

```
go
f(1, 2, 3)
// Boxes: int → interface{}, int → interface{}, int → interface{}
// 3+ allocations just for boxing
```

Drawback 4: API Ambiguity

```
go
func process(items ...string)

process()           // nil slice
process([]string{}...) // empty slice
process(nil...)     // error

// Caller confusion: which should I use?
```

Drawback 5: Inlining Issues

Compiler is less likely to inline variadic functions:

```
go
func small(x int, y ...int) int {
    return x + len(y)
}

// May not inline due to variadic parameter
```

Drawback 6: Encourages Weak Typing

```
go
```

```
func do(args ...interface{})  
// What types are allowed?  
// What's the contract?  
// No type safety!
```

Better:

```
go  
  
func doInt(values []int)  
func doString(values []string)  
// Clear types, compile-time safety
```

Part 15: When to Use Variadic (Developer Rules)

✓ Use Variadic When:

1. API ergonomics matter

```
go  
  
fmt.Println("Hello", "World") // Much nicer than passing slice
```

2. Call count is low

```
go  
  
// Called once at startup  
config.Load("config.yaml", "override.yaml", "local.yaml")
```

3. Argument count is naturally variable

```
go  
  
buildURL("api", parts...) // parts can be any length
```

4. User-facing APIs

```
go  
  
logger.Info("message", "key1", val1, "key2", val2) // Flexible
```

❌ Avoid Variadic When:

1. Hot loops

```
go

// BAD:
for i := 0; i < 1000000; i++ {
    sum(1, 2, 3) // Allocates 1 million times
}

// GOOD:
args := []int{1, 2, 3}
for i := 0; i < 1000000; i++ {
    sum(args...) // Allocates once
}
```

2. High-frequency logging

```
go

// BAD:
func trace(args ...interface{})

// GOOD:
func trace(msg string) // Pre-format outside
```

3. Performance-critical code

```
go

// BAD in game loop:
func update(entities ...Entity)

// GOOD:
func update(entities []Entity)
```

4. You control both caller and callee

```
go

// If you control both sides, just use slice:
func internal(items []Item) // No need for variadic
```

The General Rule

Prefer this:

```
go  
  
func process(items []T)
```

Over this:

```
go  
  
func process(items ...T)
```

When performance matters or calls are frequent.

Complete Mental Model

At Call Site

```
You write:      f(1, 2, 3)  
                ↓  
Compiler sees:  "variadic call"  
                ↓  
Compiler creates: tmp := []int{1, 2, 3}  
                ↓  
Compiler calls: f(tmp...)  
                ↓  
Which becomes: f(tmp) // regular call
```

Inside Function

```
Function signature: func f(nums ...int)  
                ↓  
Function receives: nums []int // regular slice  
                ↓  
You can use:     len(nums), nums[0], range nums  
                ↓  
It's just a slice: No special runtime behavior
```

Cost Analysis

Regular call: `f(slice)`

Cost: Zero allocation (just pass pointer)

Variadic call: `f(1, 2, 3)`

Cost: One allocation (create slice)

Variadic + boxing: `f(interface{ } values: 1, 2, 3)`

Cost: Multiple allocations (slice + boxing)

Summary: Core Truths

1. Variadic parameter = regular slice inside function
2. Compiler creates slice at call site (allocation cost)
3. Passing existing slice with `(...)` avoids allocation
4. Only last parameter can be variadic (grammar rule)
5. Zero arguments = `(nil)` slice
6. Cannot mix explicit and `(...)` unpacking
7. Typed `(nil)` can be unpacked, untyped cannot
8. `(...interface{})` = double allocation (slice + boxing)
9. `(append)` is special, other variadics are not
10. Can take address of slice header and elements
11. Must use `(...)` to forward variadic arguments
12. Variadic is compile-time sugar, not runtime feature
13. Hidden allocations are the main performance trap
14. Use for APIs, avoid in hot paths

Quick Reference

go

// Definition

```
func f(args ...int)    // args is []int inside
```

// Calling

```
f(1, 2, 3)            // Allocates slice
```

```
f(slice...)           // No allocation
```

```
f()                   // nil slice
```

// Forwarding

```
func g(nums ...int) {  
    f(nums...)          // Must use ...  
}
```

// Type checking

```
func h(items ...interface{}) // Expensive: boxing
```

```
func j(items ...int)         // Cheaper: no boxing
```

// Performance

// Hot path: use []int parameter

// Cold path: use ...int for ergonomics